Chunk Streaming 与卡顿问题诊断报告

Executive Summary

当前高速巡检时出现的明显卡顿，主因并不是“低模怎么还会卡”这种渲染面数问题，而是 chunk 生命周期仍然几乎全部在主线程同步完成。每当进入新 chunk，系统会立即创建 `CityChunkScene`、同步生成地形 mesh、构建 `ConcavePolygonShape3D`、生成道路 mesh、桥墩、建筑和代理节点；离开时又立即 `queue_free()`。这种“生成尖峰 + 销毁尖峰”比 LOD 过渡本身更像当前卡顿的真正来源 [1][2][3][4][5]。

除此之外，项目虽然在数据层把世界定义成 `70km x 70km`，但当前仍基于单精度全局坐标直接推进玩家和 chunk 坐标。一旦玩家真巡检到离原点数十公里的位置，精度误差会进一步放大视觉抖动、碰撞不稳定和高速移动体验问题 [6]。因此，性能与稳定性在这里是绑定问题，不应分开看。

Key Findings

- **当前 streaming 只有生命周期状态，没有真正的后台 prepare**：`CityChunkStreamer.gd` 会记录 `prepare/mount/retire` 时间，但 prepare 阶段并没有把重活提前到工作线程或 server API [1]。
- **chunk 构建尖峰在主线程同步发生**：`CityChunkRenderer.gd` 每次进入新 chunk 都直接 `CityChunkScene.new() -> setup() -> add_child()`，这会把 chunk 全量建模成本一次性打到主线程 [2]。
- **chunk 销毁也缺乏预算与复用**：退出窗口时直接 `queue_free()`，没有对象池、无分帧卸载、无冷却复用 [2]。
- **单个 chunk 的同步成本并不低**：地形 mesh、`ConcavePolygonShape3D`、道路 mesh、桥墩 MultiMesh、楼体 `StaticBody3D`、HLOD 与 occluder 都在 `CityChunkScene.setup()` 期间生成 [3]。
- **`_process()` 每帧都推进 streaming 与 HUD 统计**：这本身不是根因，但会把所有同步挂载/卸载成本更直接地暴露到帧时间里 [4]。
- **当前 `70km x 70km` 全局坐标方案与 Godot 单精度舒适区存在明显张力**：如果真到 `±35km` 位置，高速巡检下的抖动、碰撞和摄像机问题会更容易被误判成单纯“性能卡顿” [6]。

Detailed Analysis

1. 本地实现证据

`city_game/scripts/CityPrototype.gd` 在 `_process()` 中每帧调用 `update_streaming_for_position(player.global_position)`，也就是玩家位置一更新，就会驱动新的 chunk 生命周期 [4]。

接着在 `city_game/world/streaming/CityChunkStreamer.gd` 中，系统会先逻辑上标记 `prepare`，再马上进入 `mount`，最后清理 `retire`。这里虽然记录了 `last_prepare_usec`、`last_mount_usec`、`last_retire_usec`，但 prepare 阶段本身并没有真正完成“后台数据准备” [1]。

真正的同步开销发生在 `city_game/world/rendering/CityChunkRenderer.gd`： `sync_streaming()` 里只要发现新的 chunk id，就会立刻创建 `CityChunkScene` 并调用 `setup()`；离开窗口的 chunk 则直接 `queue_free()` [2]。

而 `city_game/world/rendering/CityChunkScene.gd` 的 `setup()` 又会同步完成整块地形、道路、建筑与代理内容的创建。尤其要注意的是， `GroundBody` 使用了 `ConcavePolygonShape3D`，而 Godot 官方明确把 concave/trimesh 形状视为较慢、且更适合静态环境的碰撞手段 [3][7]。

2. 当前卡顿更像“主线程尖峰”，不只是 LOD 切换

你感觉“高速行走时卡一下”，直觉上会怀疑 LOD。LOD 确实可能带来少量可见轮廓切换，但就当前代码而言，真正更重的工作是“新 chunk 被整块生成出来”。

单个新 chunk 至少包含：

- 一张 terrain mesh；
- 一套 `ConcavePolygonShape3D` 地表碰撞；
- 一套道路 ribbon mesh；
- 若干桥墩实例；
- 一批建筑 `StaticBody3D + MeshInstance3D + CollisionShape3D`；
- 中远景代理与遮挡体；
- LOD 模式初始化与统计。

这意味着巡检速度越快、跨 chunk 越频繁，主线程越容易出现明显帧尖峰。换句话说，当前的“卡”更像 **streaming mount hitch**，而不是单纯 draw call 不够低。

3. v2 文档与实现存在一处重要口径落差

`docs/plan/v2-index.md` 中，M2 把目标写成“后台准备、主线程挂载” [8]。但从当前实现看，后台准备并未真正落地，更多只是生命周期语义上的 prepare/mount 分相。

这件事很重要，因为它解释了为什么文档上看起来 streaming 已经成立，但实际手感上仍会卡。严格讲，当前实现更像：

- 数据窗口与 chunk 生命周期成立了；
- 但重活仍集中在主线程同步完成；
- 因此“逻辑上的 streaming”成立，“无尖峰的 runtime streaming”尚未成立。

4. Godot 官方资料给出的成熟经验

Godot 官方对这类问题的建议相当明确：

- 后台加载与线程安全章节都强调：`SceneTree` 不是线程安全的，重计算与资源准备应在线程中完成，再回主线程做最小化挂载 [1][2]。
- MultiMesh 文档强调重复对象应按区域拆分批量实例化，而不是到处散落大量独立节点 [5]。
- `ConcavePolygonShape3D` 类文档明确提示其成本和适用边界，不适合作为广泛动态交互的万能碰撞策略 [7]。
- Large world coordinates 文档说明，超过常规单精度范围时，应考虑 origin shifting 或双精度，以避免远离原点后的精度损失 [6]。

这和当前项目的问题是完全对齐的：不是 Godot 不能做，而是当前实现还停留在“先跑起来”的结构，没有进入“按帧预算构建 chunk”的阶段。

5. 70km 目标对性能与稳定性的额外影响

项目配置文件 `city_game/world/model/CityWorldConfig.gd` 已把世界大小设置为 `70000m x 70000m`，边界即 `±35000m`。如果玩家真的能一路巡检到这个范围附近，问题将不只是一帧里创建了多少节点，还包括：

- 摄像机抖动；
- 碰撞与贴地精度下降；
- 远距离 mesh 与物理接触点精度不稳；
- 高速移动时误把精度抖动感知为“卡顿”。

因此，针对你说的“高速行走时明显卡”，整改不能只做 streaming 分帧，还必须同时设定中长期精度路线。

Areas of Consensus

- 大场景 chunk 流式的核心是“后台准备 + 主线程轻挂载”，而不是仅有生命周期标签 [1][2]。
- 大量重复对象应优先批量实例化并按区域拆分，而不是大量独立节点 [5]。
- 静态地形的大面积 trimesh/concave 碰撞要谨慎使用，尤其在频繁创建销毁的 chunk 流程中 [7]。
- 真正的超大世界巡检需要把精度问题与 streaming 性能一起设计 [6]。

Areas of Debate

- **准备结果的表示形式**：可以在线程里预构建“纯数据 payload”，也可以直接准备离树场景；当前项目更推荐先做数据 payload，因为调试更简单。
- **chunk 复用策略**：对象池与分帧销毁都可行；短期更建议先做对象池 + mount budget。
- **精度路线**：origin shifting 的侵入性较低，双精度更彻底但需要自编译引擎与模板 [6]。

Sources

[1] Godot Engine Documentation. “Background loading.” 官方文档，说明后台线程资源准备的基本模式。
https://docs.godotengine.org/en/stable/tutorials/io/background_loading.html

[2] Godot Engine Documentation. “Thread-safe APIs.” 官方文档，说明 SceneTree 与线程边界，以及 server API 的适用范围。
https://docs.godotengine.org/en/stable/tutorials/performance/thread_safe_apis.html

[3] 本地代码证据：`city_game/world/rendering/CityChunkScene.gd`。用于确认 chunk 构建包含 terrain mesh、`ConcavePolygonShape3D`、道路、建筑、HLOD 与 occluder。

[4] 本地代码证据：`city_game/scripts/CityPrototype.gd`、`city_game/world/streaming/CityChunkStreamer.gd`。用于确认 streaming 在每帧驱动，且 prepare/mount/retire 当前仍是同步逻辑流。

[5] Godot Engine Documentation. “Optimization using MultiMeshes.” 官方文档，说明大规模重复实例的推荐表达方式。
https://docs.godotengine.org/en/stable/tutorials/performance/using_multimesh.html

[6] Godot Engine Documentation. “Large world coordinates.” 官方文档，说明单精度建议范围、双精度与 origin shifting 取舍。
https://docs.godotengine.org/en/stable/tutorials/physics/large_world_coordinates.html

[7] Godot Engine Documentation. “ConcavePolygonShape3D.” 官方类文档，说明 trimesh/concave 碰撞的性能与适用边界。
https://docs.godotengine.org/en/stable/classes/class_concavepolygonshape3d.html

[8] 本地文档证据：`docs/plan/v2-index.md`。用于确认 M2 当前声称“后台准备、主线程挂载”的验收口径。

Gaps and Further Research

- 这份报告没有实际抓取 Godot profiler、Physics frame time 或 frame capture，只完成了静态代码路径诊断。后续落地前应补一轮真实运行采样。
- 还没有比较 `HeightMapShape3D`、分块简化碰撞、道路桥面分段盒碰撞三种策略在你机器上的真实成本差异。
- 如果未来加入车辆系统，高速巡检卡顿会被进一步放大，因为道路 ahead-of-player 预取半径必须更大。